UNIVERSITÀ DEGLI STUDI DI UDINE

Laboratorio di Programmazione Web
(laurea in Scienze e Tecnologie Multimediali)

A.A. 2026/2027

Web link alle pagine del corso:

http://elearning.uniud.it/moodle/course/view.php?id=2688

Server side programming
PHP scripting language

I materiali in queste slide sono tratti in parte da:

V. Della Mea, L. Di Gaspero, I. Scagnetto,"Programmazione lato server", Apogeo.
http://www.w3.org/
http://en.wikipedia.org (Wikipedia)
http://www.php.net

The PHP scripting language

I. Overview and history

- PHP is a scripting language originally designed for web development, and especially for server-side programming.
- PHP in origin was an acronym for Personal Home Page, but it now stand for PHP: Hypertext Preprocessor
- PHP is free software released under the PHP License.
- PHP commands are usually embedded directly into an HTML source document, but can also be run on a command-line interface and to create standalone graphical applications (PHP-GTK)
- PHP can be deployed on most web servers and as a standalone command shell on almost every operating system.
- PHP's principal reference web pages: http://www.php.net/

The PHP scripting language

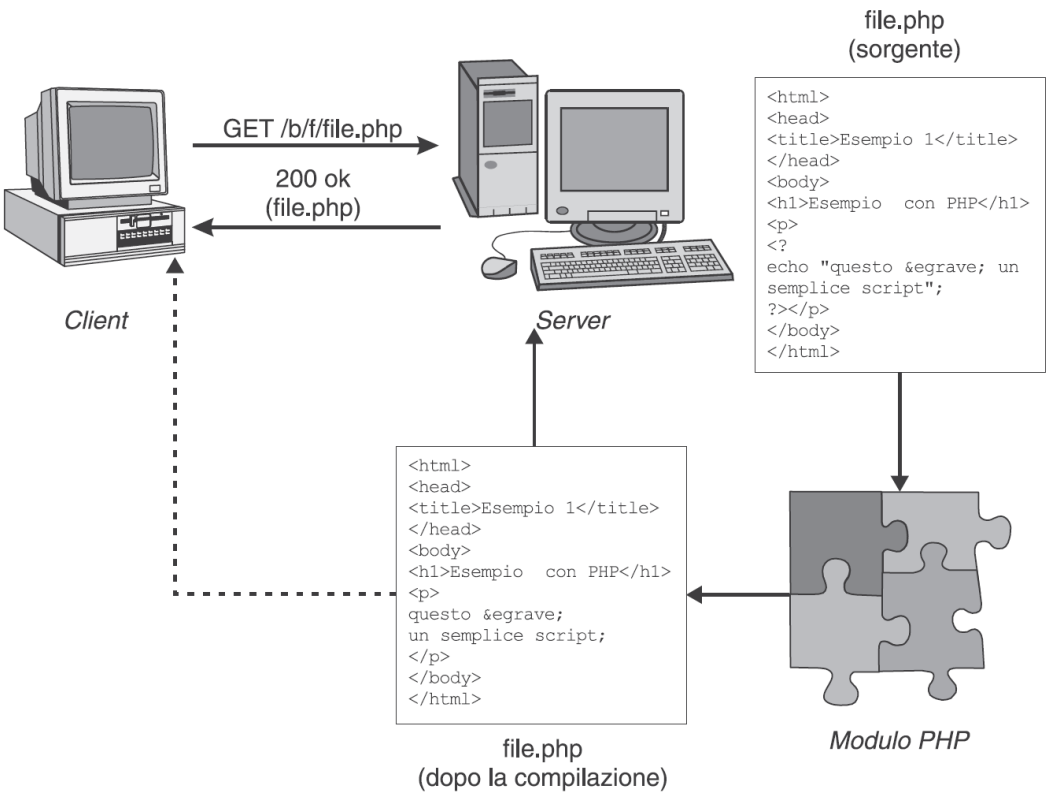
II. Server-side scripting

- The PHP script fragment interpretation process: PHP code is interpreted by a web server with a PHP processor module which generates the resulting web page.

- The PHP interpreter executes PHP code within PHP delimiters :

  <?php … ?>

```
1  <html>
2   <head>
3    <title>PHP Example</title>
4   </head>
5   <body>
6    <p>
7    <?php
8     echo "Hello World!";
9    ?>
10   </p>
11   </body>
12  </html>
```



The PHP scripting language

II. Server-side scripting: common techniques in dynamic PHP pages design

1. Include (X)HTML elements in the text printed by echo (or print) instructions in the PHP script

```
1 <?php
2  echo"<p>Di seguito ho dei commenti</p>\n"; // questo dura fino a fine riga
3  // questo occupa tutta la riga,
4  # ed anche questo.
5  // echo "<p>Questo testo non verrà mai emesso perche l'istruzione è commentata</p>\n";
6 ?>
```

1. Interleave PHP and (X)HTML code

```
1 <?php
2   $a=5; //assign a value to variable a
3 ?>
4 <p>Here some (X)HTML code</p>
5 <?php
6   // here use the previously defined variable
7   echo 'The value of $a is: ', $a;
8 ?>
```

The PHP scripting language

IV. Principal PHP programming language elements

- Instructions
- Comments
- Variables and Data Types
- Operators and expressions
- Functions
- Program flow control
- Object oriented programming support (since PHP5)

The PHP language syntax

I. Instructions

- A PHP program consists of a sequence of instructions
- Instructions must be terminated by a semicolon ";"

Es 1

```
1 <?php
2   echo 'This is a test';
3 ?>
```

Es 2

```
1 <?php echo 'This is a test'; ?>
```

Es 3

```
1  <?php echo 'We omitted the last closing tag';
```

The PHP language syntax

II. Comments

- PHP supports ‘C’, ‘C++’ and Unix shell-style comments
- for comments longer than a single line: /*comments*/
- for single – line comments: //comments or #comments

```
1  <?php
2    echo 'This is a test'; // This is a one-line c++ style comment
3
4    /* This is a multi line comment
5        yet another line of comment */
6
7    echo 'This is yet another test';
8
9    echo 'One Final Test'; # This is a one-line shell-style comment
10 ?>
```

The PHP language syntax

II. Comments

- Multi line /* */ comments cannot be indented:

```
1 <?php
2  /*
3   echo 'This is a test'; /* This comment will cause a problem */
4  */
5 ?>
```

The PHP language syntax

III. Variables

- Variables in PHP are represented by a dollar sign followed by the name of the variable. The variable name is case-sensitive.

- A valid variable name starts with a letter or underscore

```php
<?php
  $var = 'Bob';
  $Var = 'Joe';
  echo "$var, $Var";      // outputs "Bob, Joe"

  $4site = 'not allowed!';     // invalid; starts with a number
  $_4site = 'this is fine';    // valid; starts with an underscore
?>
```

- In PHP the explicit declaration of variables is not required. Everywhere in the program it is possible to define and use a new variable.

The PHP language syntax

III. Variables and the *echo* instruction

- The *echo* instruction outputs one or more strings separated by commas

```php
<?php
  echo "Hello World";

  echo "This spans
    multiple lines. The newlines will be
    output as well";

  echo "This spans\nmultiple lines. The newlines will be\noutput as well.";

  echo "Escaping characters is done \"Like this\".";

  // You can use variables inside of an echo statement
  $foo = "foobar";
  $bar = "barbaz";
  echo "foo is $foo"; // foo is foobar
?>
```

The PHP language syntax

III. Variables and the *echo* instruction

- More on the *echo* instruction

```php
<?php

  // Using single quotes will print the variable name, not the value
  echo 'foo is $foo';

  // If you are not using any other characters, you can just echo variables
  echo $foo;            // foobar
  echo $foo,$bar;       // foobarbarbaz

  // Some people prefer passing multiple parameters to echo over concatenation.
  echo 'This ', 'string ', 'was ', 'made ', 'with multiple parameters.', '\n';
  echo 'This ' . 'string ' . 'was ' . 'made ' . 'with concatenation.' . '\n';
?>
```

The PHP language syntax

III. Variables and their *scope (visibility)*

- The scope of a variable is the context in which the variable is defined, and thus it can be used
- Normally, it is the entire PHP file in which they are defined

```
1 <?php
2   $a=5; //assign a value to variable a
3 ?>
4 <p>Here some (X)HTML code</p>
5 <?php
6   // here use the previously defined variable
7   echo 'The value of $a is: ', $a;
8 ?>
```

The PHP language syntax

III. Predefined variables

- PHP provides a wide number of predefined variables, mainly:
  - used to describe the server hosting the scripts
  - used to manage the HTTP requests (using GET and POST, cookies, ecc.),
  - environment variables

```
1 <?php
2   echo"<a href=\"http://",
3   $_SERVER["HTTP_HOST"],// returns the name of the host site
4   $_SERVER["PHP_SELF"], // returns the name of the script
5   "\">Link to myself</a>\n";
6 ?>
```

The PHP language syntax

IV. Data types

- Computers are able to manipulate and store directly only sequences of bits. These can be interpreted in different ways.
- The sequence 0100 0110 can be interpreted as the integer number 70 or as the character F.
- High-level programming languages such as C, Java, PHP, assign data types to variables, ie indicates how to interpret the content.
- PHP provides eight predefined types, four of which are basic types, two are compound types, and two are special.
- PHP is not a strictly typed language: the type of a variable is inferred from the context of use, and based on this can vary.
- The type of a variable can also be explicitly changed with opportuneconversion functions.

The PHP language syntax

IV. Basic Data types

- **Booleans:** a boolean expresses a truth value. It can be either TRUE or FALSE (case-insensitive: TRUE, True, true are all correct).

```php
<?php
 $foo = True; // assign the value TRUE to the variable
?>
```

- **Integer:** an integer is a number of the set $\mathbb{Z}$ = {..., -2, -1, 0, 1, 2, ...}. Integers can be specified in decimal (base 10), hexadecimal (base 16), octal (base 8) or binary (base 2) notation, optionally preceded by a sign (- or +).

```php
<?php
  $a = 1234; // decimal number
  $a = -123; // a negative number
  $a = 0123; // octal number (equivalent to 83 decimal)
  $a = 0x1A; // hexadecimal number (equivalent to 26 decimal)
?>
```

The PHP language syntax

IV. Basic Data types

- **Floats:** a float represents a floating point naumber (approximations of real numbers). Usually encoded with the IEEE 754 double precision format using 64 bits.

```php
<?php
 $a = 1.234; // [sign]integer_part.decimal_part
 $b = 1.2e3; // exponential notation
 $c = 7E-10; // exponential notation
?>
```

The PHP language syntax

IV. Basic Data types

- **Strings**: a string is series of characters, where a character is encoded as a byte (i.e., PHP only supports a 256-character set, and hence does not offer native Unicode support).

- The PHP language provides a number of functions to process strings (concatenation, substring search, string matching, etc.)

- String literals can be specified in different ways, the principal ones being with single quotes (‘…’) and double quotes (“…”).

The PHP language syntax

V. The String data type

- To specify a string literal, one way is to enclose it in single quotes (the character ').

- Note that variables and escape sequences for special characters will not be expanded when they occur in single quoted strings (e.g., \r or \n will not be interpreted as new line or tab).

- To specify a literal single quote, escape it with a backslash (\). To specify a literal backslash, double it (\\).

```php
<?php
  echo 'this is a simple string';
  echo 'You can also have embedded newlines in
  strings this way';

  // Outputs: Arnold once said: "I'll be back"
  echo 'Arnold once said: "I\'ll be back"';

  // Outputs: You deleted C:\*.*?
  echo 'You deleted C:\\*.*?';

  // Outputs: This will not expand: \n a newline
  echo 'This will not expand: \n a newline';

  // Outputs: Variables do not expand either
  echo 'Variables do not $expand $either';
?>
```

The PHP language syntax

V. The String data type

- Another way is to enclose it in double quotes (the character ”).

- Note that variables and escape sequences for special characters will be expanded when they occur in double quoted strings (e.g., \r or \n will interpreted as new line or tab).

```
1 <?php
2   $juice = "apple";
3   echo "He drank some $juice juice.\n";
4   echo "He drank some juice made of $juices."; //invalid
5 ?>
```



He drank some apple juice.
He drank some juice made of .

The PHP language syntax

V. The String data type

```
1 <?php
2  $a='stringa compresa tra apici singoli';
3  $b='stringa con l\'apice, e un \na-capo';
4  $c="stringa compresa tra doppi apici o virgolette";
5  $d="tra virgolette possiamo inserire anche altri caratteri speciali, come la
6  \t tabulazione, e anche variabili ($a) che vengono espanse";
7  echo "$a\n $b\n $c\n $d\n. "
8 ?>
```

The PHP language syntax

V. The String data type

- Common escaped characters used in double quotes enclosed strings.

Escaped characters

Sequence	Meaning
\n	linefeed (LF or 0x0A (10) in ASCII)
\r	carriage return (CR or 0x0D (13) in ASCII)
\t	horizontal tab (HT or 0x09 (9) in ASCII)
\v	vertical tab (VT or 0x0B (11) in ASCII) (since PHP 5.2.5)
\e	escape (ESC or 0x1B (27) in ASCII) (since PHP 5.4.0)
\f	form feed (FF or 0x0C (12) in ASCII) (since PHP 5.2.5)
****	backslash
\$	dollar sign
\"	double-quote
\[0-7]{1,3}	the sequence of characters matching the regular expression is a character in octal notation
\x[0-9A-Fa-f]{1,2}	the sequence of characters matching the regular expression is a character in hexadecimal notation

The PHP language syntax

VI. Arrays

- Arrays are used to represent data collections
- Arrays are made of elements sequences which can be indexed through a numerical index (starting from 0)
- In PHP, arrays can be also represented through associative maps. A map is a type that associates values to keys.
- Array values can be other arrays (trees and multidimensional arrays) .
- An array can be created using the array() language construct .

```php
<?php
 $v1 =array();// an empty array
 $v2 =array("Monday","Tuesday","Wednesday");
 echo $v2[2];// prints Wednesday, the element in cell 2
?>
```

The PHP language syntax

VI. Arrays

- The array() construct accepts any number of comma separated key => value pairs as arguments for associative arrays construction.
- Keys can be integer numbers or strings, and values can be any valid type.
- Access to values is achieved through index or associative key within square brackets [ ].

```php
<?php
 $v3 =array(
 0=>"Monday",
 1=>"Tuesday",
 2=>"Wednesday");
?>
```

```php
<?php
 $v4=array(
 "apple"=>"fruit",
 "mango"=>"fruit",
 "artichoke"=>"vegetable");
 // this is an associative array
 echo $v4["mango"];
 //prints the value associated
 //with mango, i.e fruit
?>
```

The PHP language syntax

VI. Arrays

- In PHP the type of elements can be heterogeneous.
- The count() instruction takes an array variabile and returns the number of elements in the array.

```php
<?php
 // an array can have other arrays as elements
 // and these can be heterogeneous
 $automobili =array(
 "Paperone" =>array(1, 213),
 "Paperino" => 313,
 "Qui"=> "nessuna" );
 echo count($automobili); // prints 3
?>
```

The PHP language syntax

VI. Arrays

- More on manipulating arrays

```php
<?php
 $v5 =array(
  3=>"Thursday",
  4=>"Friday",
  5=>"Sunday");
  // array in which first index is 3
  $v5[5] = "Saturday"; //changes a value
  $v5[] = "Sunday";//appends a value to the array
  $v5[1] = "Tuesday"; // adds an element at position 1
?>
```

The PHP language syntax

VI. Arrays

- More on manipulating variables/arrays: destroy a variable or remove elements from arrays

```php
<?php
// destroy a single variable
unset($foo);

// destroy a single element of an array
unset($bar['quux']);

// destroy more than one variable
unset($foo1, $foo2, $foo3);
?>
```

The PHP language syntax

VI. Arrays

- How to print an array:
  - `echo` is intended for scalar values, only returns “Array” when used to print an array
  - `print_r` recursively prints an array, showing all keys and values:

```
$a = array ('a' => 'apple', 'b' => 'banana', 'c' => array ('x', 'y', 'z'));
echo array($a); // returns "Array"
print_r($a); // returns
Array
(
    [a] => apple
    [b] => banana
    [c] => Array
        (
            [0] => x
            [1] => y
            [2] => z
        )
)
```

The PHP language syntax

VII. Global and superglobal variables

- *Superglobals* are built-in variables that are always available in all scopes
- Superglobals are used to access data from the server running the scripts or from external HTTP requests.
- Several predefined variables in PHP are "superglobals“: there is no need to declare them as *global* to access them within functions.
- Examples of superglobal PHP variables are:

  $_SERVER (Server and execution environment information)
  $_GET (variables passed to the current script via the URL parameters)
  $_POST (variables passed to the current script via the POST method)
  $_COOKIE
  $_SESSION
  …

The PHP language syntax

VIII. Data exchange through web pages and PHP scripts

- Server-side data processing requires that data are made available from remote host through the HTTP protocol.
- HTTP client-server communication scheme imply that data can be sent to the server only during the page request process.
- HTTP provides the GET and POST methods to this aim.

The PHP language syntax

VIII. Data exchange through web pages and PHP scripts

- With the GET method, the data to be sent to the server must be encoded in the URI string. Usually, this method is used to send short, non sensible, text data.
- E.g.: http://myserver.com/test.php?firstname=Mario&lastname=Rossi
- Data sent through the GET method will be available in the superglobal $_GET variable.
- $_GET is an **associative array** having as keys the name of data (*firstname* and *lastname*), and the sent values (*Mario* and *Rossi*) as values

The PHP language syntax

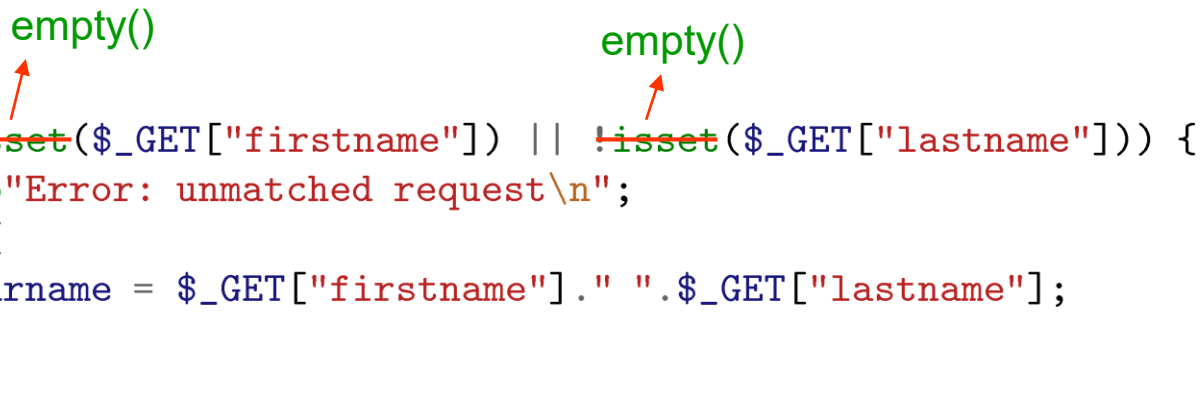
VIII. Data exchange through web pages and PHP scripts

- The handling PHP script on the server will access the sent data through the $_GET variable:

```php
<?php
  // print the whole content of array S_GET (to be used for debug only)
 print_r($_GET);

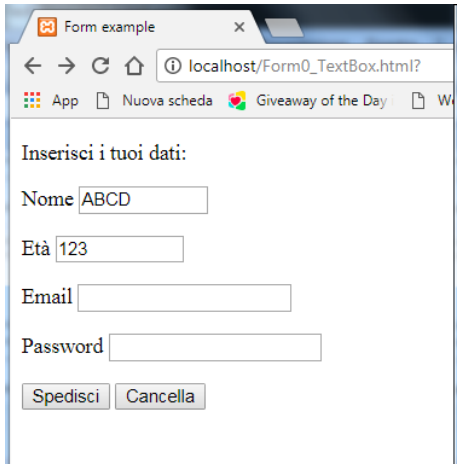
 $yourname = $_GET["firstname"]." ".$_GET["lastname"];
>
```

- Or (safer):



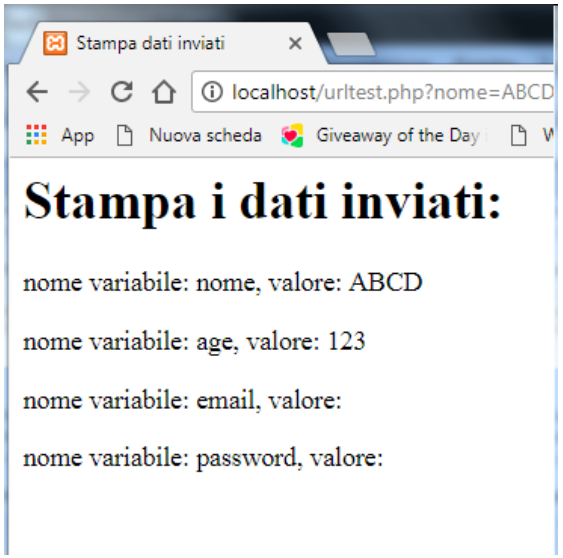
The PHP language syntax

VIII. Data exchange and variable checking in PHP scripts: *isset()* vs *empty()*



invio

```
<p><h1>Stampa i dati inviati:</h1></p>
<?php
foreach($_GET as $K => $V){
    echo "<p>nome variabile: $K, valore: $V</p>";
}
?>
```



isset(): true
empty(): false

isset(): true
empty(): true

VIII. More on the *empty()* function

However: empty() treats the integer 0, the string "0", the float 0.0, empty arrays [], null, and false as empty, returning true.

- According to the manual, it
  «Returns **true** if var does not exist or has a value that is empty or equal to zero, aka falsey, see conversion to boolean. Otherwise returns false.»
  https://www.php.net/empty

If you need zero (0 or "0") to be treated as a valid, non-empty value, you must replace empty() with more explicit strict checks.

- E.g.,: check isset and not empty string:

```
if (isset($var) && $var !== '') {
    // "0" and 0 will pass as not empty
}
```

VIII. Data exchange through web pages and PHP scripts

- With the POST method, the data to be sent to the server is encoded in the HTTP message body. This allows to send non-text data (e.g., binary)

- This method requires to collect the data through a form-based web page.

- E.g.:

```
1 <form action="script.php" method="post" >
2  <p>
3   Dato 1:<input type="text" name="var1" />
4   <input type="text" name="var2" />
5   <input type="hidden" name="var3" value="valore" />
6   <input type="submit" />
7  </p>
8 </form>
```

The PHP language syntax

VIII. Data exchange through web pages and PHP scripts

1. The handling PHP script on the server will access the sent data through the $_POST variable:

```php
<?php
  echo"<p>Primo valore:", $_POST["var1"], "</p>\n";
  echo"<p>Secondo valore:", $_POST["var2"], "</p>\n";
  echo"<p>Valore nascosto :", $_POST["var3"], "</p>\n";
>
```

